

Integrated Food–Grocery Delivery ABM Documentation

Simulation procedure, model specification baseline, agent flowcharts, and interaction matrix

Current-code edition: 19 September 2026

The Manhattan matching, tariff distance, four-direction movement and Store–Restaurant–Customer route remain the model baseline. This edition documents the paired one-factor-at-a-time controller that is currently implemented: integration probability, food preparation, grocery preparation and Store-selection policy are varied while the reference demand and fleet remain 300 food Customers and 12 Drivers. Daily demand and fleet size can be supplied for an entire experiment, but are not sensitivity grids in the current controller. The document describes implemented behavior, not empirical validation or platform policy. Export-recovery findings from 8 September 2026 and the 13 September review snapshot are retained as historical evidence in Appendices A and B.

Table 1: Notation and current values used in the simulation procedure

Notation	Meaning	Current value / setting
s	Configuration for one model invocation.	p_I (current default 0.44), seed z , demand CSV, daily count Q and fleet size M .
S	Collection of experimental configurations.	Controller builds one-factor-at-a-time conditions and matching $p_I = 0$ controls.
R_{rep}	Independent paired day replications.	Controller default 30; at least two for an interval.
Z	Experimental seed schedule.	Consecutive integers starting at 42 by default; each seed defines one paired day block.
z	Seed active in the model.	Default 42; nonnegative integer.
T, t	Horizon and domain tick.	$T = 1440$; $t = 0, \dots, 1439$; 1 tick = 1 minute.
τ	Unassigned-order timeout.	23 ticks; evaluated before matching.
E	Spatial environment.	Non-toroidal 50×50 Mesa MultiGrid.
$\mathcal{D}$	Set of Driver agents.	$ \mathcal{D} = M$; reference $M_0 = 12$.
$\mathcal{R}$	Set of Restaurant agents.	$ \mathcal{R} = 50$.
$\mathcal{G}$	Set of Store agents.	$ \mathcal{G} = 15$.
c, o	Customer and its unique Order.	Q Customers/day; reference $Q_0 = 300$; one food-containing order each.
I_o, p_I	Integration indicator and probability.	$I_o \sim \text{Bernoulli}(p_I)$; $p_I \in [0, 1]$; current CLI default 0.44.
b_o	Creation tick of order o .	Timestamp used in $t - b_o$.
d, m	Activated Driver and first pickup Merchant.	Restaurant for food-only; Store for integrated.
$D(a, b)$	Manhattan distance in orthogonal grid steps.	$ x_a - x_b + y_a - y_b $.
$D_{d,o}$	Matching distance to first Merchant.	$D(d, m)$, then FIFO, then order ID.
(x_a, y_a)	Coordinates of agent a .	Integers 0–49 on each axis.
L_o	Billable distance in km.	Merchant-to-Customer legs only; see §1.16.
C_o^F, C_o^G	Food and grocery item values in rupiah.	Nonnegative integer draws; $C_o^G = 0$ for food-only.
α_D	Platform share of the delivery fee.	Fixed at 0.08 for every completed order.
α_M	Platform share of gross Merchant proceeds.	Baseline assumption 0.1781025273, common to Restaurants and Stores.
N_d^{move}	Actual cell changes by Driver d .	Includes travel to the first pickup.

Algorithm 1: Integrated Food–Grocery Delivery Simulation Procedure

This algorithm describes one model invocation. The separate `simulation_experiments.py` executes the outer loop over paired day blocks and the configured one-factor-at-a-time conditions in §7. The named stages below correspond to `IntegratedDeliveryModel.step`; the domain clock is `tick_counter`, not Mesa’s internal step count. The listed random draws apply to a standalone invocation; a controller run reads equivalent values from its pre-drawn scenario. Pseudocode line numbers are local to this edition, not Python line numbers.

Line	Procedure
1:	Input: p_I (current default 0.44), seed z (default 42), demand CSV, Q (default 300), M (default 12).
2:	Validate configuration; initialize model RNG using z and non-toroidal MultiGrid.
3:	Initialize and uniformly place Drivers, then Restaurants, then Stores.
4:	Initialize empty Customer, order-history, active-order and KPI collections.
5:	Read CSV bytes once, record SHA-256, validate and aggregate hourly weights.
6:	Build 1440-slot demand schedule totaling Q ; record current PDF hash if present.
7:	for $t = 0, \dots, 1439$ do
8:	Generate Customers and Orders
9:	for each scheduled Customer at t do
10:	Create Customer at a uniformly drawn grid position.
11:	Draw $I_o \sim \text{Bernoulli}(p_I)$; choose Restaurant uniformly.
12:	If integrated, independently choose Store uniformly.
13:	Draw food preparation duration, then food item value.
14:	If integrated, draw grocery preparation duration, then grocery item value.
15:	Create component records in <code>PREPARING</code> ; attach them to their Merchants.
16:	Create one <code>AVAILABLE</code> order; link Customer, food, and optional grocery.
17:	Calculate Manhattan billable distance, delivery-fee quote, tax, and customer-payment quote.
18:	Retain order in both historical and active order collections.
19:	end for
20:	Update Customers
21:	for each active Customer do
22:	Read its shared order status and assignment reference.
23:	If assignment reference exists, set Customer to <code>IN_SERVICE</code> .
24:	Otherwise, if order is <code>AVAILABLE</code> and $t - b_o \geq 23$:
25:	Set order <code>CANCELLED</code> , record cancellation tick, set Customer <code>EXIT</code> .
26:	If order is <code>DELIVERED</code> :
27:	Set order <code>COMPLETED</code> , record completion tick, set Customer <code>EXIT</code> .
28:	end for
29:	Update all Stores, then all Restaurants, in initialization order
30:	for each Merchant and each of its component records do
31:	Skip components already <code>HANDED_OVER</code> or <code>CANCELLED</code> .
32:	if the shared order is <code>CANCELLED</code> then
33:	If component is <code>PREPARING</code> , record <code>PREPARATION_STOPPED</code> .
34:	Otherwise record food disposal or grocery cancellation/return, by kind.
35:	Set component <code>CANCELLED</code> , record tick; continue to next component.
36:	end if
37:	If <code>PREPARING</code> and $t \geq b_o + p_{m,o}$, set <code>READY</code> and record ready tick.
38:	If all handover guards in §1.12 hold:
39:	Set component <code>HANDED_OVER</code> and record handover tick.
40:	end for
41:	Derive Merchant activity and cancellation KPIs from component records when read.
42:	Update Drivers
43:	Draw a new permutation of all Drivers from the model RNG; save activation IDs.
44:	for each Driver in that permutation do
45:	Remember its initial state; execute exactly one of the following branches.
46:	if initial state is <code>IDLE</code> then
47:	Search active <code>AVAILABLE</code> orders; if none, end this activation.
48:	Rank by $(D_{d,o}, b_o, \text{order_id})$; immediately claim the minimum.
49:	Set order <code>ASSIGNED</code> , assignment tick and Driver reference.

```
50:        Set Driver's current order; increment busy ticks by one.
51:        Enter GROCERY_TRAVEL if integrated, otherwise FOOD_TRAVEL.
52:        End this activation without moving.
53:    else
54:        Increment busy ticks by one.
55:        if initial state is GROCERY_TRAVEL or FOOD_TRAVEL:
56:            Choose a feasible four-direction neighbor/current cell by squared Euclidean score, then  $(x, y)$ .
57:            Move if cell changes; increment Driver and corresponding order-leg movement counts.
58:            If target is reached, enter corresponding pickup state and record arrival tick.
59:        else if initial state is GROCERY_PICKUP:
60:            If grocery is HANDED_OVER, enter FOOD_TRAVEL; otherwise add one grocery wait tick.
61:        else if initial state is FOOD_PICKUP:
62:            If food is HANDED_OVER, enter DELIVER; otherwise add one food wait tick.
63:        else if initial state is DELIVER:
64:            Use the same four-direction Euclidean rule toward Customer; count an actual movement if made.
65:            If Customer location is reached, require all components HANDED_OVER.
66:            Then set order DELIVERED, record tick, set Driver IDLE, clear current order.
67:            Retain the order's historical assigned-Driver ID for later settlement.
68:        end if
69:        end if; do not execute a newly entered state in this activation.
70:    end for
71:    Resolve completed and cancelled orders
72:    for each active order with terminal global status do
73:        If COMPLETED and not settled, post Driver, Merchant and Platform revenues once.
74:        Clear a Driver's current order only if it still equals this order's ID.
75:        Remove Customer from grid, active Customer registry and Mesa agents.
76:        Remove order from active collection; retain order and component histories.
77:    end for
78:    Record outputs: calculate cumulative KPI row for  $t$  and  $t + 1$  elapsed ticks.
79:    Append KPI row; increment domain clock; stop running when elapsed ticks reach 1440.
80: end for
81:    Return summary. CLI exports seven files; direct run() does not export.
82:    Do not drain, extend the horizon, or force unfinished orders to terminal states.
```

1 Current Model Specification Baseline

This section preserves the original subsection structure while resolving implementation choices against the reviewed code. Fixed baseline constants, required experiment inputs, and unimplemented extensions are distinguished explicitly.

1.1 Simulation Time and Activation Order

One tick represents one minute. The fixed horizon is $T = 1440$: tick 0 is 00:00, tick 60 is 01:00, and tick 1439 is 23:59. After the last tick, active orders remain active; there is no drain, forced cancellation, or revenue settlement for unacknowledged deliveries.

The exact order is: generate Customers/Orders; activate Customers; activate all Stores; activate all Restaurants; activate a randomly permuted sequence of Drivers; resolve terminal orders; record KPIs; increment `tick_counter`. Customer and Merchant iteration follows insertion order. The NumPy model RNG generates a new Driver permutation every tick, even if no orders are available.

The default seed is 42. In a standalone invocation, initialization places Drivers, Restaurants and Stores before drawing minute-slot remainders. One shared model RNG drives placements, integration, Merchant selection, Normal draws, demand allocation and Driver shuffling. In a paired experiment, these inputs and tick-specific Driver permutations come from the same pre-drawn day for every condition in one replication. The same seed alone does not guarantee matched inputs in standalone runs or identical results across different input bytes or dependency versions.

Customers cancel at the exact timeout before Drivers can match in that tick. Merchants act before Drivers arrive, so arrival at tick t cannot receive handover until at least $t + 1$. Customer acknowledgement likewise follows delivery by one tick, provided that next tick exists.

1.2 Spatial Environment

The environment is a non-toroidal 50×50 `MultiGrid`, with coordinates 0–49 and 2,500 cells. Multiple agents may occupy one cell. Each agent position is drawn uniformly over the grid when the agent is created. Restaurants and Stores remain stationary and permanent; a Customer remains at its delivery location until removal. There are no obstacles, road network, congestion, or cell-capacity constraints.

1.3 Agent Population

Agent	Reference count	Lifecycle / composition
Driver	$M_0 = 12$	Configurable positive integer M ; permanent for the full horizon; one operational order at a time.
Restaurant	50	Permanent and stationary.
Store	15	Permanent and stationary; count is an explicit modeling assumption because the food-delivery records do not supply it.
Customer	$Q_0 = 300/\text{day}$	Configurable positive integer Q ; generated dynamically; exits after <code>COMPLETED</code> or <code>CANCELLED</code> .

Orders and Merchant–Order components are dataclass records, not Mesa agents. Merchant records persist after handover; Customer removal does not delete order history.

1.4 Demand Generation

The daily demand is Q food Customers (default 300); there is no separate grocery stream. The default file is `dataset/food_hourly_demand_profile.csv`, resolved relative to the project root. Columns `hour` and `avg_demand` are required. All hours 0–23 must be present with no other hours; hours must be finite integers. Weights must be finite and nonnegative, with at least one positive value. Other columns, including `day_name`, are not used in the calculation.

For each hour h , average `avg_demand` across all its rows, then normalize:

$$\bar{w}_h = \frac{1}{n_h} \sum_{j: \text{hour}_j=h} w_j, \quad q_h = \frac{\bar{w}_h}{\sum_{k=0}^{23} \bar{w}_k}.$$

The default CSV contains seven days. Custom CSVs need not contain exactly seven days or equal row counts per hour. The implementation first divides weights by their maximum to avoid overflow; this common scaling leaves the normalized shares unchanged.

Allocate Qq_h using floors and the largest-remainder method; equal remainders favor the earlier hour. For hourly count n_h , assign $\lfloor n_h/60 \rfloor$ Customers to every minute and add one to $n_h \bmod 60$ distinct minute slots selected by the model RNG. The schedule always has 1440 entries summing to Q , and multiple Customers may spawn in one tick.

1.5 Food-Only and Integrated Orders

Each Customer creates exactly one Order, containing food and optionally grocery:

$$I_o \sim \text{Bernoulli}(p_I), \quad p_I \in [0, 1].$$

The current centralized default is $p_I = 0.44$; setting it to `None` makes the CLI input mandatory. The value is an experiment setting rather than an empirically calibrated estimate. $p_I = 0$ creates only food-only orders; $p_I = 1$ creates only integrated orders. A Restaurant is selected uniformly from all 50 Restaurants. For integrated orders, the standalone/default rule selects a Store using the pre-drawn uniform index. Controller conditions can instead select the Store nearest to the Restaurant or nearest to the Customer using Manhattan distance with stable Store-order tie breaking. Selection does not model capacity, inventory, acceptance or price choice. In controller runs, a fixed uniform variate U_o is stored for each potential order and $I_o(p_I) = \mathbf{1}\{U_o < p_I\}$. Potential Store and grocery attributes are pre-drawn for pairing, but are not instantiated as demand, components, revenue or travel when $I_o = 0$. The $p_I = 0$ condition therefore contains no independent grocery orders.

FOOD_ONLY: Restaurant $\rightarrow$ Customer,

INTEGRATED: Store $\rightarrow$ Restaurant $\rightarrow$ Customer.

Integration is decided at creation; it never combines two independently generated orders. The Driver also travels from its current position to the first Merchant, a leg included in cost and emissions but excluded from the delivery-fee distance.

1.6 Order and Merchant–Order Statuses

The global lifecycle is

AVAILABLE $\rightarrow$ ASSIGNED $\rightarrow$ DELIVERED $\rightarrow$ COMPLETED,

with the alternative AVAILABLE $\rightarrow$ CANCELLED. Component statuses are PREPARING, READY, HANDED_OVER and CANCELLED. A component can be cancelled from PREPARING or READY; handed-over records are terminal. These are distinct from the Merchant’s derived agent state. `active_orders` contains available, assigned and delivered orders; `orders` retains all history.

1.7 Driver Matching / Check Order

There is no Dispatcher. Each activated IDLE Driver searches all active AVAILABLE orders and accepts its selected order deterministically. The first pickup Merchant m is the Restaurant for food-only and the Store for integrated orders. Rank candidates lexicographically by

$$(D_{d,o}, b_o, \text{order_id}), \quad D_{d,o} = |x_d - x_m| + |y_d - y_m|.$$

Assignment updates shared status immediately, so a later Driver in the same tick cannot claim that order. Ties are resolved by creation tick (FIFO), then smallest order ID. Claiming consumes the activation; travel begins on a later tick.

1.8 Driver Movement

In a travel branch, evaluate the four orthogonal neighbors and current cell (Von Neumann neighborhood), retaining positions inside the non-toroidal grid. For current position (x, y) , candidates are $(x - 1, y)$, $(x + 1, y)$, $(x, y - 1)$, $(x, y + 1)$ and (x, y) . Choose the candidate p minimizing the squared Euclidean score to target t :

$$Q(p, t) = (x_p - x_t)^2 + (y_p - y_t)^2.$$

Coordinate tuple (x, y) breaks a tie deterministically. Squaring preserves the Euclidean minimizer, so no square root or additional random draw is needed. Move at most one orthogonal step; diagonal movement is not permitted. Q selects direction only and is never accumulated as traveled kilometers.

$$N^{\text{move}}(a, b) = |x_a - x_b| + |y_a - y_b|, \quad 1 \text{ actual step} = 0.5 \text{ km}.$$

For an unobstructed leg, this is the movement count, not total service duration. Claim, state transitions, pickup and acknowledgement introduce additional activations. Even a zero-distance leg uses its travel branch to recognize arrival. Every actual orthogonal step uses the same modeled length, consistent with the conceptual 30 km/h speed. Stationary activations add no travel distance.

For example, $(0, 0) \rightarrow (3, 4)$ requires seven moves: 3.5 km, Rp1,400 operating cost and 420 emission model units. From $(49, 49)$ toward $(0, 0)$, equal Euclidean scores are resolved to $(48, 49)$. This is an unobstructed orthogonal-road abstraction, not a measured street network. The former diagonal baseline required four moves for the first example. Changing the baseline can change matching, travel/service duration, tariff and delivered-order counts; no legacy mode is exposed.

1.9 Driver States and State Execution

The Driver states are **IDLE**, **GROCERY_TRAVEL**, **GROCERY_PICKUP**, **FOOD_TRAVEL**, **FOOD_PICKUP**, and **DELIVER**. Only the branch active at the beginning of an activation executes. A Driver can hold one operational order; it clears **current_order_id** immediately upon delivery and becomes idle. The delivered Order retains **assigned_driver_id** as historical attribution.

At the next tick the Driver may claim another order before the previous order is resolved. Resolution clears a Driver's current reference only if it still points to the order being resolved, protecting a new assignment. Historical assignment IDs are never erased during normal resolution.

1.10 Preparation Time

Draw once per component, independently of other component draws:

$$p_{R,o} = \lceil \max\{0, \mathcal{N}(12.9028672628, 4.0083718821^2)\} \rceil, \quad p_{G,o} = \lceil \max\{0, \mathcal{N}(31, 19^2)\} \rceil.$$

Durations are in ticks. Planned readiness is $b_o + p_{m,o}$. If not cancelled, a **PREPARING** component becomes ready on the first Merchant activation with $t \geq b_o + p_{m,o}$. Duration zero permits readiness in the creation tick. **ready_tick** is the observed event; **planned_ready_tick** remains defined even for a cancelled component.

1.11 Multiple-Order Merchant Representation

The Merchant agent state is derived from its records:

$$s_m = \begin{cases} \text{ACTIVE}, & \text{any component is PREPARING or READY,} \\ \text{IDLE}, & \text{otherwise.} \end{cases}$$

Preparation progresses in parallel with no capacity limit. All Merchants are visited each tick; their terminal component records are skipped. The conceptual methods **CheckOrders**, **ProcessOrders**, **CheckDrivers**, **HandleHandover** and **HandleTimeout** are responsibilities, not separately exposed Python methods. Their actual implementation is mainly **Merchant.step**, **_can_handover** and **_handle_timeout**. Cancellation is handled before readiness and handover, as shown in Figure 3.

1.12 Handover Conditions

A handover requires all of the following:

- Component status is **READY** and global order status is **ASSIGNED**.
- The assigned Driver exists and its current order ID matches this order.
- Driver and Merchant occupy the same actual grid cell.
- Driver state is **GROCERY_PICKUP** at a Store or **FOOD_PICKUP** at a Restaurant.
- For integrated food, grocery is already **HANDED_OVER**.

Merchant activation posts the handover. Later in that tick, the Driver pickup branch sees the flag and changes state without moving. Arrival alone is insufficient because the Merchant has already acted earlier in the arrival tick.

1.13 Customer Behavior

Customer states are **WAITING**, **IN_SERVICE** and **EXIT**. A Customer reads its shared Order status and assignment reference; it does not inspect Driver coordinates or preparation progress. Assignment is first observed on the Customer activation following the claim. A delivered order becomes completed on the next Customer activation. This records the completion tick, sets **EXIT**, and triggers later settlement/removal in the same domain tick.

1.14 Timeout and Cancellation

Cancellation occurs only when

$$t - b_o \geq 23, \quad d_o = \emptyset, \quad u_o = \text{AVAILABLE}.$$

Assigned orders never timeout, even if total service time exceeds 23 minutes. Customer cancellation precedes Merchant readiness checks. If cancellation coincides with planned readiness while status is still **PREPARING**, the event is preparation stopped.

Status when cancelled	Recorded event	Accounting treatment
PREPARING (either kind)	PREPARATION_STOPPED	One cancelled unit and full component item value in Merchant cancellation KPIs.
READY food	PREPARED_FOOD_DISPOSAL	Disposal event; excluded from PREPARING cancellation KPIs.
READY grocery	GROCERY_CANCELLATION_RETURN	Cancellation/return event; excluded from PREPARING cancellation KPIs.

For Merchant m , define the historical set $\mathcal{C}_m^{\text{stop}}$ of components with event **PREPARATION_STOPPED**. Then

$$U_m^{\text{cancel}} = |\mathcal{C}_m^{\text{stop}}|, \quad V_m^{\text{cancel}} = \sum_{c \in \mathcal{C}_m^{\text{stop}}} \text{item_value}_c.$$

The exported fields are **preparing_cancelled_units** and **preparing_cancelled_product_value**. One food/grocery component counts as one unit, including a zero-value product. Value is rupiah before tax, fees and commissions. Food and grocery components are evaluated independently. Repeated reads or exports do not increment the values; they are derived from terminal events. Restaurant and Store aggregates are separate.

These are product-value accounting KPIs, with no payer, compensation payment, refund, inventory replenishment or revenue/profit posting. **READY** cancellation events remain records, not an inventory or waste-cost model. Compensation is not an unresolved active field in standalone schema 7 or paired-scenario schema 8. The timeout is 23 minutes, while grocery preparation has mean 31 minutes, so **PREPARING** cancellation KPIs are expected to be especially relevant for unassigned integrated orders.

1.15 Price Generation

Food and integrated grocery item values are

$$C_o^F = \lceil \max\{0, \mathcal{N}(33760, 4418^2)\} \rceil, \quad C_o^G = \lceil \max\{0, \mathcal{N}(42491, 29634^2)\} \rceil.$$

For food-only orders $C_o^G = 0$. Clamping negative draws and taking the ceiling are modeling conventions, not calibrated empirical facts. Within creation, food duration/value are drawn before grocery duration/value.

1.16 Tax, Customer Payment, and Delivery Fee

Let r_o , g_o , and c_o be Restaurant, Store, and Customer positions. The implemented billable distance is

$$L_o = 0.5[D(r_o, c_o) + I_o D(g_o, r_o)] \quad \text{km},$$

where D is Manhattan distance in grid steps and the Store term is used only for integrated orders. Driver-to-first-Merchant travel is excluded. This is a quote calculated at order creation from stationary endpoint coordinates.

$$C_o = C_o^F + C_o^G, \quad \text{Tax}_o = 0.10C_o, \quad F_o^{\text{del}} = \max\{9000, 2250L_o\},$$

$$P_o = C_o + \text{Tax}_o + F_o^{\text{del}}.$$

There is no ceiling to whole billed kilometers. Distances through 4 km are covered by the Rp9,000 minimum; for example, $L_o = 2.5$ gives Rp9,000 and $L_o = 5$ gives Rp11,250. Rp9,000 and Rp2,250/km are the midpoints of the Zone I ranges in Indonesian Ministry of Transportation Decision KP 667/2022. They are used as a generic app-based motorcycle-service proxy, not as a verified GrabFood or other platform tariff. `customer_payment` is a quote, including on unfinished or cancelled orders; `settled_customer_payment` sums quotes only for settled completed orders.

1.17 Driver Operating Cost and Emission

At Rp400/km and 0.5 km/orthogonal step, each actual cell change costs Rp200. Emission is 60 model units per actual step, with no physical mass unit inferred. Costs and emissions include travel to the first pickup and all later legs, even for orders unfinished at the horizon. Waiting and stationary activations generate neither. Thus

$$\text{distance}_d = 0.5N_d^{\text{move}}, \quad \text{cost}_d = 200N_d^{\text{move}}, \quad \text{emission}_d = 60N_d^{\text{move}}.$$

1.18 Revenue and Profit

For each completed order and Merchant component, the implemented equations remain

$$G_{m,o} = (1 + 0.10)C_o^m, \quad R_{d,o} = (1 - \alpha_D)F_o^{\text{del}} = 0.92F_o^{\text{del}},$$

$$R_{m,o} = (1 - \alpha_M)G_{m,o} = 0.8218974727G_{m,o} = 0.90408722C_o^m,$$

$$R_{p,o} = P_o - R_{d,o} - \sum_{m \in M_o} R_{m,o}, \quad \Pi_d = \sum_o R_{d,o} - 200N_d^{\text{move}}.$$

The Driver share is fixed at 92 percent for every order, so the Platform retains 8 percent of the delivery fee. This implements the study's constant-share policy using Grab Indonesia's statement that its Driver-partner commission is at most 8 percent from 1 July 2026; the model deliberately applies the maximum uniformly and does not implement transaction-level exceptions. See the Grab Indonesia announcement.

Grab's public Merchant terms confirm that a service fee is deducted but place its percentage in Merchant-specific commercial terms rather than publishing one universal GrabFood/GrabMart rate. The common $\alpha_M = 0.1781025273$ is therefore an explicit modeling assumption for both Restaurants and Stores, chosen to preserve the preceding specification's effective settlement while replacing its composite expression with one auditable share of gross Merchant proceeds. It is not presented as an official Grab tariff. See the GrabMerchant terms. Platform revenue remains the residual of customer payment after Driver and Merchant shares; it is revenue, not Platform profit. Settlement occurs once in resolution after `COMPLETED`; a guard prevents repeated posting. The historical assigned Driver receives the credit even if it has claimed another order. An order delivered at tick 1439 remains unacknowledged and receives no revenue in that run.

Money is calculated with `Decimal` and no explicit intermediate rounding; display formatting alone rounds. JSON serializes `Decimal` values as strings and CSV as decimal text. The engine computes Driver profit; it does not model Merchant or Platform costs or their profits. These equations remain modeling assumptions requiring independent empirical validation.

1.19 Defined KPI Outputs

At the end of each tick, cumulative KPIs are stored with the tick label and elapsed-tick count. `summary()` can also be read before any tick or during a partial run. The full field catalog is in §6.

Let $a_o, d_o^{\text{time}}, f_o, x_o$ denote assigned, delivered, completed and cancelled ticks, respectively. Timestamp differences and eligible observations are:

Metric	Definition	Included in observed mean when
Assignment waiting	$a_o - b_o$	Assignment tick exists.
Delivery service	$d_o^{\text{time}} - b_o$	Delivery tick exists, even if not completed.
Acknowledgement delay	$f_o - d_o^{\text{time}}$	Delivery and completion ticks exist.
Completion total	$f_o - b_o$	Completion tick exists.
Cancellation waiting	$x_o - b_o$	Cancellation tick exists.
Food/grocery pickup waiting	Unsatisfied pickup activations	Corresponding handover tick exists.

Pickup waiting counts each Driver activation already in a pickup state that has not received handover. It excludes the arrival activation and successful handover activation. The `*_so_far` fields also retain waits for unfinished pickups. Each time mean is paired with `observed_<metric>_count`. No eligible observations yields CSV blank / JSON `null`, not zero.

A busy tick is an activation that begins non-idle or successfully claims an order. At e elapsed ticks,

$$u_d = \frac{B_d}{e}, \quad u_{\text{fleet}} = \frac{\sum_d B_d}{|\mathcal{D}|e};$$

both are unobserved at $e = 0$. Claim, travel, pickup and delivery activations count as busy, including zero-distance travel.

`delivered_orders` is the current DELIVERED status count; `delivered_cumulative` includes all delivery events, including completed orders. The reconciliation identities are

$$\begin{aligned} \text{generated} &= \text{available} + \text{assigned} + \text{delivered} + \text{completed} + \text{cancelled}, \\ \text{active} &= \text{available} + \text{assigned} + \text{delivered}, \\ \text{delivered cumulative} &= \text{delivered} + \text{completed}, \\ \text{settled payment} &= \text{Driver revenue} + \text{Merchant revenue} + \text{Platform revenue}. \end{aligned}$$

Cancellation units/value, disposal, return and preparation-stopped events are cumulative. Terminal components remain in records. Current component-status counts and current active-Merchant counts are separate from those historical event measures.

1.20 Open Parameters and Policies

The empirical value p^* , reference daily count Q_0 , reference fleet size M_0 , three sensitivity grids, replication count and seed start remain experiment-design inputs. One engine invocation accepts one p_I , Q , M and seed; the controller executes batches without changing the established Store-first route. Merchant selection, CSV mapping and integrated tariff distance are implemented choices, described above. Compensation payments, behavioral rejection, grocery-only demand, batching, congestion and Merchant capacity are outside this implementation. Tax, prices, durations, cost and emissions still require empirical calibration where research conclusions depend on them.

1.21 Consistency of the Native-LaTeX Flowcharts and Interaction Matrix

Figures 1–3 and Table 2 are native TikZ/LaTeX. Driver matching is Claim Order, without a Dispatcher. Merchants inspect the assigned Driver and react to Customer cancellation. Merchant cancellation precedes readiness/handover. Driver delivery and Customer acknowledgement are distinct events. Diagram action labels summarize responsibilities rather than promising separate methods. Customer tracking does not expose component preparation progress.

1.22 Baseline Interpretation

This is a reviewed implementation baseline, not a declaration that every implemented behavior is desirable. Changes to mechanisms, activation, inputs, accounting or versions should be compared explicitly with this edition. The matrix in Appendix C records discrepancies and their disposition; Appendix A records known software defects without altering the scripts. Updating this PDF changes its hash for newly constructed models; existing simulation exports retain their original provenance.

2 Agent Flowcharts

Rectangles denote states; parallelograms denote actions; diamonds denote decisions; dashed parallelograms denote initialization/spawn. An arrow to a new state does not execute that state in the same activation. Re-enter the applicable branch on the next tick.

2.1 Driver Agent Flowchart

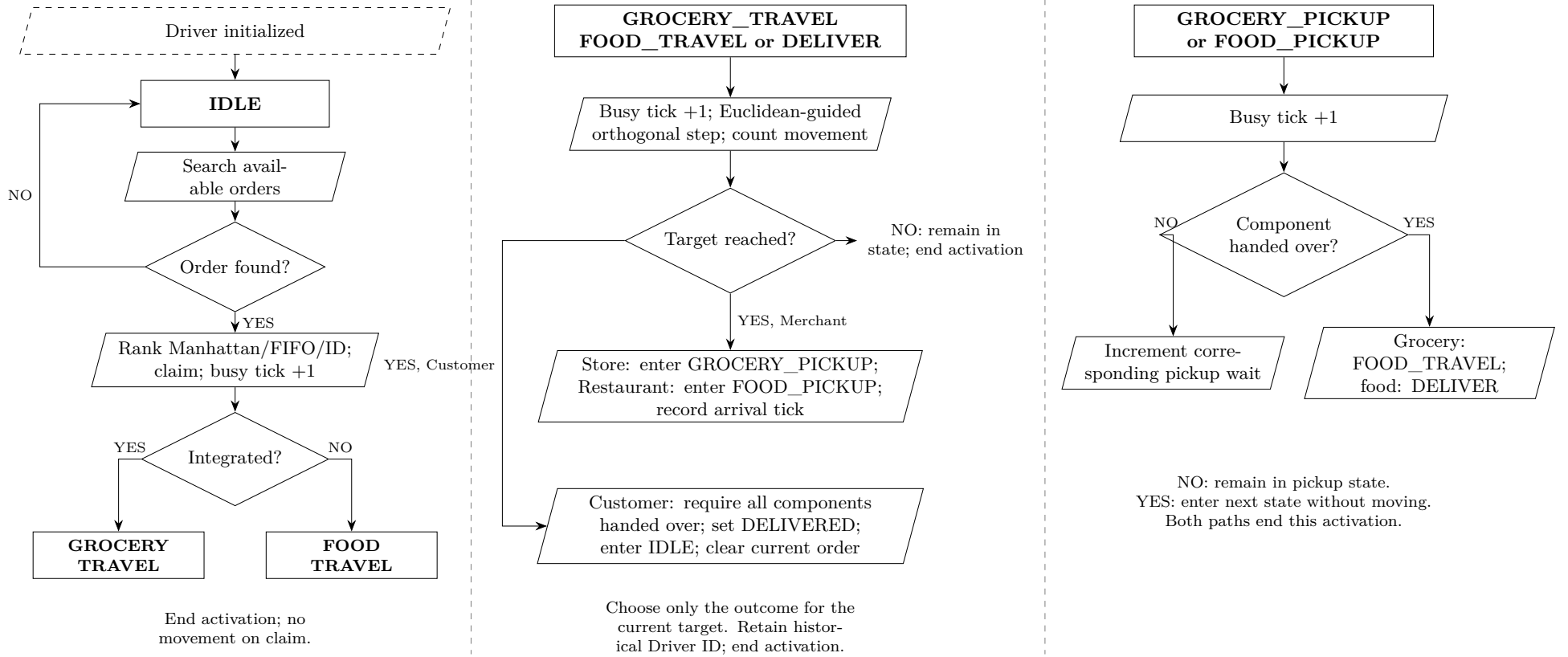


Figure 1: Driver Agent Flowchart. Choose the panel from the initial state of this tick's activation. Panels describe mutually exclusive branches, not a same-tick sequence. Matching is Driver-owned; there is no Dispatcher.

2.2 Customer Agent Flowchart

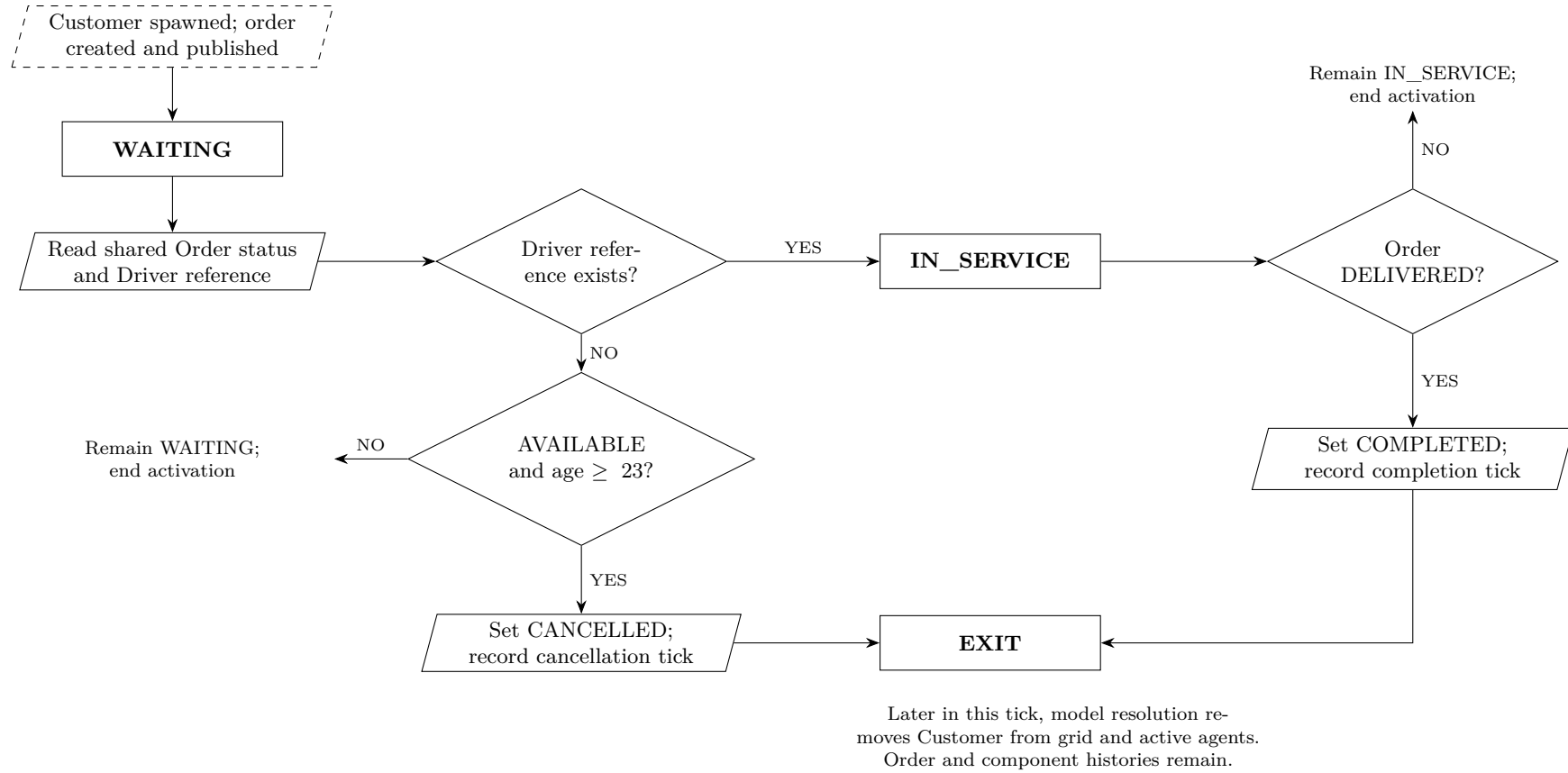


Figure 2: Customer Agent Flowchart for reachable engine states. Assignment and delivery are observed through shared Order data. No Driver-position or component-preparation observation is performed. A delivered order retains its Driver reference, so it follows the assigned branch before completion.

2.3 Merchant Agent Flowchart

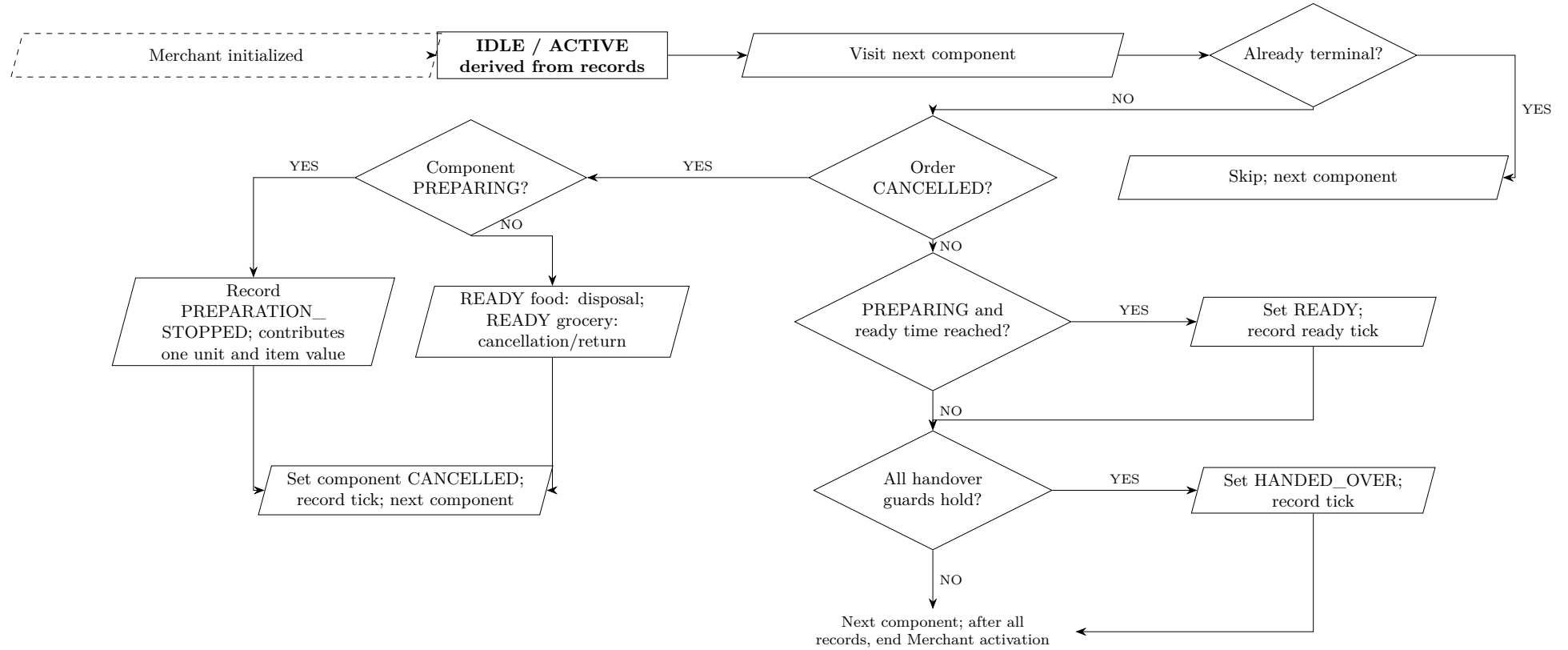


Figure 3: Merchant Agent Flowchart, shared by Store and Restaurant. Terminal means HANDED_OVER or CANCELLED. Cancellation is evaluated before preparation readiness. The guards are READY component, ASSIGNED order, matching Driver/current order, co-location, correct pickup state, and grocery precedence for integrated food. ACTIVE means any PREPARING/READY component; the engine visits even IDLE Merchants.

3 Agent Interaction Matrix

Rows identify the acting/source agent; columns identify the affected agent or the owner of data being observed. Shared Order references are included. These are logical interactions, not network messages. Creation/attachment is performed by the model on the Customer's behalf.

Action from	Customer	Driver	Restaurant	Store
Customer	—	Publishes AVAILABLE order; exposes cancellation/completion in shared status. Reads assignment reference.	Food component attached at creation. Shared cancellation and completion status becomes visible.	Grocery component attached for integrated order. Shared cancellation and completion status becomes visible.
Driver	Claims order; sets ASSIGNED. Delivers and sets DELIVERED; Customer observes on its next activation.	—	Claim links Driver/order; physical arrival and FOOD_PICKUP state are observable by Restaurant.	Claim links Driver/order; physical arrival and GROCERY_PICKUP state are observable by Store.
Restaurant	No direct preparation-progress observation by Customer; processing updates the component record.	Reads assigned Driver/current order, position and state; hands over READY food if guards hold.	—	Reads grocery HANDED_OVER flag through shared Order as a food-handover prerequisite.
Store	No direct preparation-progress observation by Customer; processing updates the component record.	Reads assigned Driver/current order, position and state; hands over READY grocery if guards hold.	Publishes grocery HANDED_OVER flag, which Restaurant can read through shared Order.	—

Table 2: Agent Interaction Matrix aligned with the implemented shared-Order architecture.

Interpretation. Check Drivers is an information read, not dispatch. Customer cancellation affects only unassigned orders, so Merchant timeout does not release an assigned Driver. Restaurant–Store entries describe a one-way shared component-status dependency, not direct communication. Model-level settlement credits Driver, Merchant and Platform revenue after Customer completion; Platform is an account, not a Mesa agent. Model resolution removes Customers and active orders while retaining history.

4 Visualizer and Playback

4.1 Architecture and Display Semantics

`GridVisualizer` uses `Matplotlib` to display an existing `IntegratedDeliveryModel`. Drawing reads detached `snapshot()` data; only playback callbacks call `step()`. Snapshot, rendering, Pause and Speed consume no model RNG draws. One advancing frame represents one domain tick regardless of wall-clock interval. A GUI-capable `Matplotlib` backend is needed for an interactive window; headless `Agg` is used for automated validation.

The grid has equal aspect and displays coordinates 0–49. Restaurants are brown squares, Stores green triangles, active Customers purple diamonds, and Drivers colored circles. Merchant outlines are dark when any component is PREPARING or READY; otherwise they are white. Driver styles are:

State	Color	Meaning
IDLE	Gray (#777777)	Available to search/claim.
GROCERY_TRAVEL	Blue (#0072B2)	Travelling to Store.
GROCERY_PICKUP	Light blue (#56B4E9)	Awaiting/observing grocery handover.
FOOD_TRAVEL	Vermilion (#D55E00)	Travelling to Restaurant.
FOOD_PICKUP	Orange (#E69F00)	Awaiting/observing food handover.
DELIVER	Purple (#CC79A7)	Travelling to Customer.

Driver IDs are shown by default. For co-located agents, display positions use deterministic radius-0.25 sub-cell offsets ordered by agent ID. This changes only artist coordinates; model positions, distances and handover checks remain integer-cell based.

Solid blue segments connect Driver display position to the active target. Dashed gray segments connect remaining stops. They show straight connections between targets, not a history of actual four-direction grid movements or a detailed planned street route. A diagonal connector does not authorize a diagonal Driver step. Stops use real grid coordinates, which can differ slightly from offset Merchant/Customer marker positions.

4.2 Dashboard Coverage

The clock displays the last executed tick (initial state before any tick, then 00:00 through 23:59) and elapsed ticks out of 1440. The dashboard displays generated/type/status counts; completed service units and productivity; delivered cumulative; food/grocery preparing and ready counts; disposal and return events; average assignment, delivery, acknowledgement and pickup waits; fleet utilization; distance, total emission and emission intensity; Driver cost, revenue and profit; and aggregate Merchant/Platform revenue.

The dashboard does *not* display all exported KPIs. In particular, PREPARING cancelled units/product value, completion-total and cancellation-wait means, observation counts, Restaurant/Store revenue splits, and per-agent records remain available in snapshot/exports rather than dedicated dashboard lines. A dash represents an unobserved time value; formatting rounds the display only.

4.3 Playback Controls and State

Construction starts paused in READY unless the supplied model is finished. `run()` starts animation automatically. The visible run labels are READY, RUNNING, PAUSED and FINISHED; they are playback labels rather than Order statuses.

Control / event	Implemented behavior
Pause / Resume	Stops/starts the animation event source without advancing the model.
Step	Pauses playback, advances at most one tick, refreshes, and remains paused. At the horizon, no additional tick runs.
Speed	Slider 0.25x–10x, increments 0.25x; interval becomes $\max(1, \lfloor \text{interval_ms}/\text{speed} \rfloor)$ ms. Model time per tick remains one minute.
Reset	Pauses, constructs a fresh model with the same config/seed and the same experiment scenario if present, verifies demand-byte hash, exports the previous run if needed, then replaces the model. Remains paused.
Horizon reached	Stops animation, sets FINISHED, and attempts export. No drain.
Window close	Stops playback and attempts one export of current state. Does not run remaining ticks.

Reset checks the candidate input *before* exporting/replacing the previous model. Changed CSV bytes (even just an added newline), unreadable input, or construction error block reset, preserve the current model, and show a footer message. Export failure also preserves the model and blocks reset. The candidate reads the current PDF anew; Reset verifies the demand hash, not the old PDF hash or dependency versions.

With an explicit output directory, reset runs use sibling names `<name>_reset_1`, `<name>_reset_2`, etc. The first successful reset creates index 1. Default destinations are unique per export. A successful automatic export is cached for that run to avoid duplicate exports. Expected export failures are printed and shown in the footer; the CLI exits with code 1 if an export error remains after `run()`. Incomplete-write and close-retry limitations are detailed in Appendix A; export success is not guaranteed merely because a close/reset action was requested.

5 Python and Command-Line Interfaces

5.1 Python Interface

Entry point	Contract
<code>SimulationConfig(p_integration, seed=42, demand_csv=DEFAULT_DEMAND_CSV, daily_customers=300, num_drivers=12)</code>	Frozen dataclass. Probability finite real in $[0,1]$, excluding booleans. Seed nonnegative integer, excluding booleans. Daily count and Driver count positive integers, excluding booleans. CSV path expanded and resolved.
<code>IntegratedDeliveryModel(config, scenario=None)</code>	Constructs agents, schedule and provenance; no domain tick executed. Optional scenario supplies a shared exogenous day; the standalone call uses its model RNG. Reads CSV and, if available, PDF bytes.
<code>build_experiment_scenario(seed, demand_csv, daily_customers=300, num_drivers=12)</code>	Creates one pre-drawn day with potential food/grocery inputs and 1440 Driver permutations.
<code>model.step()</code>	Executes one tick; no-op after horizon. Returns no summary.
<code>model.run()</code>	Continues from current state to horizon and returns summary dictionary. Does not export.
<code>model.summary()</code>	Current cumulative KPI dictionary plus FINISHED/PARTIAL state and horizon; does not advance.
<code>model.snapshot()</code>	Detached display dictionary containing dimensions, clocks, config values, Drivers/routes, Merchants, active Customers and KPIs. Does not advance or draw RNG.
<code>model.export_results(output_dir=None)</code>	Writes seven files and returns resolved destination Path. Can export an empty or partial run. Requires a new or empty destination.
<code>GridVisualizer(model, interval_ms=50, show_driver_ids=True, show_routes=True, output_dir=None)</code>	Creates figure/controls. Positive interval required. Holds supplied model; <code>run()</code> enters Matplotlib playback.

Internal underscore-prefixed callbacks are implementation details rather than a supported user control API. Exter-

nal mutation of model agents, schedules or component records is used in controlled tests, not validated production input.

```
from src.simulation_engine import SimulationConfig, IntegratedDeliveryModel
model = IntegratedDeliveryModel(SimulationConfig(p_integration=0.5, seed=42))
model.step() # Executes domain tick 0.
view = model.snapshot() # Reads detached data.
summary = model.run() # Continues to tick 1439, no drain.
destination = model.export_results()
The probability 0.5 is an example override; the centralized current default is 0.44.
```

5.2 Command-Line Arguments

Both scripts share the following arguments, plus standard `-h/--help`:

Flag	Default / constraint	Meaning
<code>--p-integration</code>	Required; finite 0–1	Probability for creation of integrated orders.
<code>--seed</code>	42; nonnegative integer	Model RNG seed.
<code>--daily-customers</code>	300; positive integer	Total daily food-containing orders.
<code>--num-drivers</code>	12; positive integer	Permanent fleet size for this run.
<code>--demand-csv</code>	Project’s default demand CSV	Custom hourly profile; columns/validation in §1.4.
<code>--output-dir</code>	Unique directory under <code>output/simulations</code>	Explicit new or empty destination; no overwrite.

Visualizer-only arguments are `--interval-ms` (positive integer; default 50), `--hide-driver-ids` and `--hide-routes` (both false by default). The standalone engine and visualizer accept daily count and Driver count; grid size, Merchant counts, timeout and horizon remain fixed. For automatic multi-replication, the separate controller has its own CLI in §7.

```
conda activate abms
python -B src/simulation_engine.py --p-integration 0.5
python -B src/simulation_visualizer.py --p-integration 0.5
python -B -m unittest discover -s tests -p "test_simulation*.py" -v
```

Run these examples from the project root. The default CSV is independent of working directory; a user-supplied relative CSV or output path resolves from the invoking working directory. Invalid parsed configuration/input or expected CLI I/O errors are reported via argparse. See Appendix B for actual tested versions, which differ from some declared pins.

6 Exports, KPI Catalog, and Reproducibility

6.1 Seven-File Export Contract

Table 3: Exported files (standalone schema 7; paired-scenario schema 8)

File	Granularity	Contents
<code>kpi_ticks.csv</code>	End of each executed tick	Cumulative KPI row, clock, Driver activation order, agent-state and component-status counts, time means/counts, cancellation units/value, travel and finance.
<code>orders.csv</code>	Every generated order	Type, IDs, status, positions, timestamps, durations, quote and settlement, movement legs, cost/emission/profit, activity at export.

File	Granularity	Contents (continued)
<code>components.csv</code>	Every food/grocery component	Parent/merchant IDs, kind, value, preparation duration, planned/observed readiness, handover/cancellation ticks, event, settled revenue, parent status.
<code>drivers.csv</code>	Each permanent Driver	Final position/state/current order, job counts, movement, cost/emission, busy ticks/utilization, ongoing wait totals, revenue/profit.
<code>merchants.csv</code>	Each permanent Merchant	Position/kind/state, received/current/completed component counts, cancellation events, PREPARING units/value, revenue.
<code>summary.json</code>	Current whole run	Current KPIs; <code>finished</code> , <code>run_status</code> , <code>horizon_ticks</code> , <code>financial_note</code> .
<code>metadata.json</code>	Run provenance	Schema, timestamp, Python/libraries, config/baseline, demand and PDF hashes, optional paired-scenario hash, policies, metric definitions, hourly weights/counts, full minute schedule, elapsed/finished.

One uninterrupted day yields 1440 KPI rows (ticks 0–1439), Q order rows, Q plus the integrated-order count component rows, M Driver rows, and 50 Merchant rows. Partial exports have only observed rows, including zero order/component/KPI rows before the first tick; CSV headers remain present. Every permanent Merchant remains, including those with no received components.

6.2 KPI and Record Field Families

Names below are literal field names or explicitly marked expansion patterns. The formulas and observation rules in §1.19 apply to both tick rows and current summary values.

Family	Fields / interpretation
Clocks	<code>tick</code> , <code>elapsed_ticks</code> ; summary before the first tick has null tick and zero elapsed ticks.
Order counts	<code>generated_orders</code> , <code>food_only_orders</code> , <code>integrated_orders</code> ; <code>available_orders</code> , <code>assigned_orders</code> , <code>delivered_orders</code> , <code>completed_orders</code> , <code>cancelled_orders</code> ; <code>delivered_cumulative</code> , <code>active_orders</code> , <code>active_customers</code> .
Productivity	<code>completed_food_service_units</code> , <code>completed_grocery_service_units</code> , <code>completed_service_units</code> ; <code>service_units_per_driver_hour</code> , <code>service_units_per_busy_driver_hour</code> , and <code>service_units_per_km</code> . One completed food and one completed grocery component are equal, unweighted units.
Emission	<code>emission_units</code> is total model emission; <code>emission_intensity_per_service_unit</code> divides it by completed service units and is null when none are complete. Units are not calibrated CO ₂ e.
Food protection	<code>food_completion_rate</code> , <code>food_cancellation_rate</code> , <code>food_unfinished_rate</code> ; mean and P90 food completion, P90 food pickup wait, and mean/P90 food post-pickup delivery time. All generated orders contain food.
Integrated experience	Completion, cancellation and unfinished rates restricted to integrated orders; mean/P90 integrated completion and mean/P90 food and grocery pickup waits. Values are null when no integrated orders exist.
Travel / finance	<code>movement_count</code> , <code>travel_distance_km</code> , <code>driver_operating_cost</code> , <code>emission_units</code> , <code>driver_revenue</code> , <code>restaurant_revenue</code> , <code>store_revenue</code> , <code>merchant_revenue</code> , <code>platform_revenue</code> , <code>driver_profit</code> , <code>settled_customer_payment</code> .
Driver activity	<code>driver_busy_ticks</code> , <code>driver_utilization</code> , <code>driver_activation_order</code> (comma-separated ID string); <code>drivers_<state></code> for all six lowercase Driver state names.
Components / Merchants	<code><kind>_<status>_components</code> expands food/grocery and preparing/ready/handed_over/cancelled; <code>active_restaurants</code> , <code>active_stores</code> . These are current status counts over retained history, not just active orders.
Cancellation events	<code>food_disposal_events</code> , <code>grocery_return_events</code> , <code>preparation_stopped_events</code> .

Family	Fields / interpretation (continued)
Cancellation KPIs	<code><merchant>_preparing_cancelled_units</code> and <code><merchant>_preparing_cancelled_product_value</code> , with <code>merchant =</code> restaurant/store.
Time means / counts	For each metric below, <code>mean_<metric></code> and <code>observed_<metric>_count</code> : <code>assignment_wait_ticks</code> , <code>delivery_service_ticks</code> , <code>acknowledgement_delay_ticks</code> , <code>completion_total_ticks</code> , <code>cancellation_wait_ticks</code> , <code>food_pickup_wait_ticks</code> , <code>grocery_pickup_wait_ticks</code> .
Waits so far	<code>food_pickup_wait_ticks_so_far</code> , <code>grocery_pickup_wait_ticks_so_far</code> ; includes ongoing waits, unlike the observed pickup means.

Order rows preserve `assigned_driver_id` historically; `active_at_export` marks membership in the active collection. `first_pickup_moves`, `store_to_restaurant_moves` and `restaurant_to_customer_moves` sum to movement count. Arrival timestamps are distinct from component handover timestamps. `settled`, `settlement_tick`, `driver_revenue`, `platform_revenue`, and component `settled_revenue` distinguish posted revenue from `customer_payment`, `tax`, `delivery_fee` and `billable_distance_km` quotes.

Driver `assigned_order_count` uses historical IDs; `delivered_order_count` uses delivery-event existence; `completed_order_count` uses completed status. Merchant `completed_components` counts settled parent orders, not merely handed-over components. Merchant `received_components` and event counts are historical; agent `ACTIVE` is derived only from preparing/ready records.

6.3 Encoding and Storage Rules

Money is Decimal text in CSV and a JSON string. Missing observations are CSV blanks / JSON null; zero is retained for observed zero duration, zero-valued items, and zero counts. Structured paths/config metadata are read at model construction except for export time/library versions. CSV uses UTF-8, a header row, and normal CSV quoting (including activation-order strings).

Default destinations use UTC timestamp plus a UUID fragment under the project output directory. A supplied destination must be new or empty; existing results are never intentionally overwritten. Seven files are written sequentially, so an I/O failure can leave a partial directory (review finding R1). Export does not advance the model or consume its RNG.

6.4 Schema and Provenance

Standalone `metadata.json` declares `schema_version:7`; experiment-scenario runs declare version 8 and include `experiment_scenario` seed and hashes, plus the shared-random-input policy. Version 7/8 adds the two-dimensional service-level fields to the productivity and emission measures from the preceding schema; older exports are not migrated or rewritten. Both retain PREPARING-only Merchant cancellation KPIs without payment postings. Compare `policies.matching`, `policies.billable_distance`, `policies.movement` and the specification PDF hash to identify the model used. Current policies record Manhattan matching to the first Merchant, Manhattan billed legs, and four-direction/current-cell movement scored by squared Euclidean distance then (x, y) . The Python helper is `manhattan(a, b)`; the former `chebyshev` helper has no compatibility alias. The daily-count and Driver-count flags extend the standalone CLI. Metadata config includes both values, and the historical baseline object records the actual run population. The current controller manifest declares `experiment_schema_version:8`.

Demand provenance includes resolved path, SHA-256 of exact bytes, column mapping, aggregation, normalized weights, hourly allocation and minute schedule. PDF provenance includes path and SHA-256 captured at model construction, or null if the file was absent. Export records Python and installed library versions and an export timestamp. No engine/visualizer source hash is included in simulation metadata; Appendix B separately identifies reviewed source bytes.

Reproduction requires the same implementation, configuration, input bytes and compatible dependency behavior. With these held fixed, engine history and visualized playback match; timestamp, unique destination name and absolute paths are not simulation outcomes. A PDF rebuild changes its byte hash without changing the engine, so subsequent metadata can differ even if simulation history does not. The engine does not read model parameters from PDF text.

7 Paired Experiment Controller

7.1 Research Contrast and Experimental Unit

The controller `simulation_experiments.py` evaluates the food-only condition $p_I = 0$ against an integrated condition $p_I = p^* > 0$ supplied explicitly or taken from the centralized default. The current centralized reference is $p^* = 0.44$, $Q = 300$ daily food orders and $M = 12$ Drivers. Grocery exists only as a component of an integrated food order; there is no independent grocery-demand stream. The route remains Store $\rightarrow$ Restaurant $\rightarrow$ Customer. The controller can override Q and M for the whole batch, but it does not construct demand or fleet sensitivity grids.

One independently seeded day block is the replication unit. For replication r , use $z_r = z_0 + r - 1$ and construct one immutable exogenous scenario X_r containing creation ticks, food and potential grocery attributes, initial agent positions, integration uniforms and 1440 Driver activation permutations. Every condition in that replication uses the identical scenario and a fresh model state:

$$I_{ro}(p) = \mathbf{1}\{U_{ro} < p\}, \quad Y_{rj}(c) = \text{day-level KPI } j \text{ under condition } c \text{ using } X_r.$$

Shared uniform values make integrated-order sets nested across probability conditions. Potential grocery attributes are ignored at $p = 0$. The realized integrated count is stochastic; it is not forced to equal Qp . All conditions sharing r are dependent paired observations; only the R_{rep} day blocks are independent. Orders and ticks are not independent replications. The scenario generator identifier stored in the manifest is currently `factorial-v1`; this is an implementation/version label for separated random streams, not a statement that the active controller runs a factorial design.

7.2 One-Factor-at-a-Time Sensitivity Design

Without `--sensitivity`, the controller runs the standard food-only baseline and the reference integrated condition. With `--sensitivity`, one factor changes at a time while all other factor values remain at their reference settings:

Factor	Current levels	Interpretation
Integration probability	0.22, 0.44, 0.66	Uptake of the grocery component; $p = 0$ is the food-only control, not a sensitivity level.
Food preparation multiplier	0.75, 1, 1.25	Scales each pre-drawn food duration and rounds upward to a whole tick.
Grocery preparation multiplier	0.75, 1, 1.25	Scales each pre-drawn grocery duration and rounds upward to a whole tick.
Store selection	uniform, nearest Restaurant, nearest Customer	Compares the pre-drawn Store index with two Manhattan-nearest rules.

The reference values p^* , multiplier 1 and uniform Store selection are always present. Food preparation changes the food-only outcome as well, so each nonreference food multiplier receives its own matching $p = 0$ baseline. The other integrated alternatives use the standard baseline. With current defaults there are nine integrated conditions and three controls per replication, hence 12 runs per replication and 360 runs for 30 replications. Preparation multipliers and alternative Store rules are exploratory, not calibrated empirical estimates. This is a sensitivity design, not an optimization procedure and not a formal interaction experiment.

7.3 Replication, Contrasts and Uncertainty

For each integrated condition c and KPI j , calculate the paired difference

$$d_{rj}(c) = Y_{rj}(c) - Y_{rj}(b(c)),$$

where $b(c)$ is the standard food-only baseline except for food-preparation conditions, which use the baseline with the same food multiplier. Let $\mathcal{O}_j(c)$ contain replications with both KPI values observed. Then

$$\bar{d}_j(c) = \frac{1}{|\mathcal{O}_j(c)|} \sum_{r \in \mathcal{O}_j(c)} d_{rj}(c).$$

The primary comparison is p^* versus zero at the supplied Q, M . The controller reports pointwise percentile 95% intervals using 10,000 bootstrap resamples of complete replication pairs separately for every condition and metric.

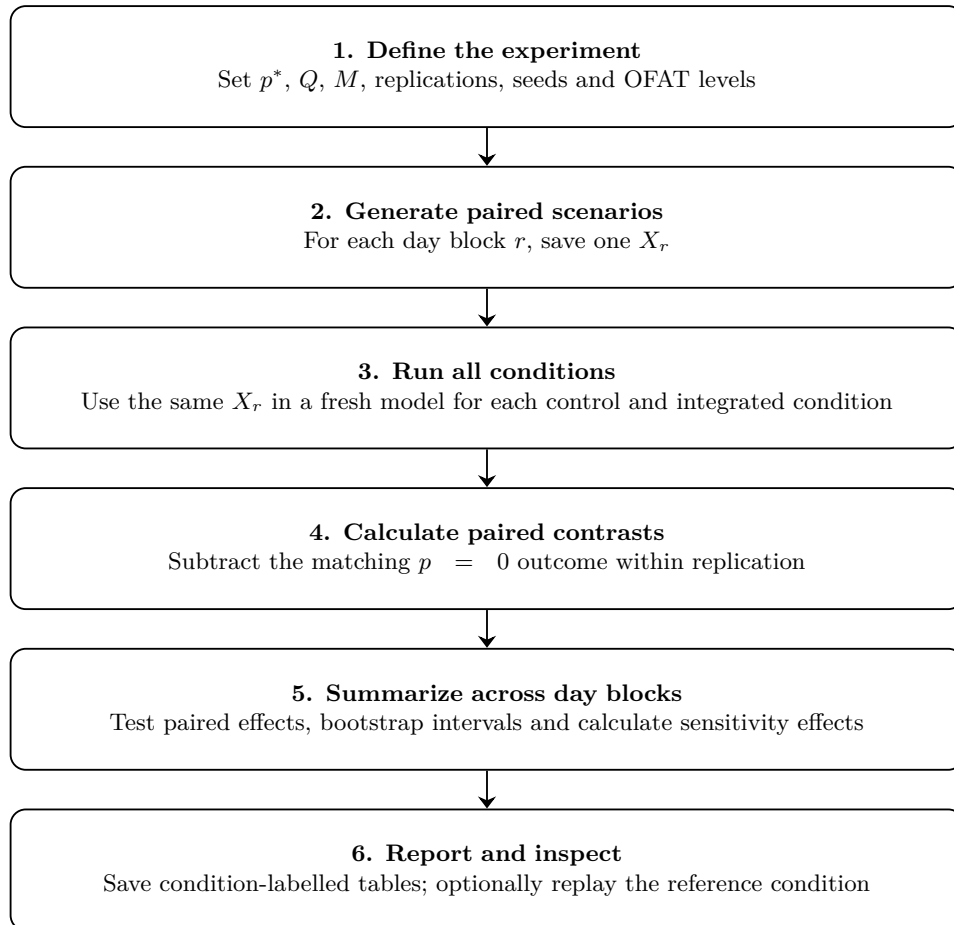
It also tests the two-sided null hypothesis that the paired effect is zero at $\alpha = 0.05$. Shapiro–Wilk is applied to the replication-level differences; failure to reject normality selects a paired t -test, while rejection selects the Wilcoxon signed-rank test. At least three observed pairs are required. Raw p -values are reported for all eligible metrics, and Holm adjustment is applied across the designated primary metrics within each condition. Paired Cohen’s $d_z = \bar{d}/s_d$ is reported when $s_d > 0$. Missing conditional-time contrasts are omitted and `observed_pairs` is reported; an interval is absent with fewer than two observed pairs. The default 30 replications is not a precision guarantee. Bootstrap intervals remain pointwise rather than simultaneous intervals. These tests address statistical significance, not a non-inferiority margin or threshold of practical importance.

For sensitivity interpretation, the controller also calculates $d_{rj}(c) - d_{rj}(c^*)$, where c^* is the reference integrated condition. The resulting `sensitivity_effects.csv` isolates the change in the paired integration effect, including the matching-baseline adjustment for food preparation.

7.4 Analysis Procedure and Flowchart

The outer controller follows these steps; each model invocation follows Algorithm 1:

1. Validate p^* , factor levels, Q , M , replication count, seed start and demand file; de-duplicate levels and include reference values.
2. For each day block r , generate and save one X_r and its hash.
3. Run every control and integrated OFAT condition in a fresh 1440-tick model using that scenario.
4. Subtract the appropriate $p = 0$ baseline within the same replication; retain missing time contrasts.
5. Summarize each condition across day blocks, bootstrap whole available pairs, run paired significance tests with Holm correction for primary metrics, and calculate sensitivity effects relative to c^* .
6. Optionally replay the selected replication under the standard baseline or reference integrated condition, without adding an observation.



All conditions for one r are evaluated before advancing to the next day block. Visualization supplies no additional replication.

7.5 KPI Interpretation

The research baseline is food-only. Integration adds a grocery component to a subset of food orders in order to test whether the same fleet and operating horizon can complete more service output. One food and one grocery are equal, unweighted service units. A service unit is counted only when its parent Order is COMPLETED; a handed-over component is not yet a completed customer service. Therefore

$$N_S = N_F + N_G = N_C + N_{C,I}, \quad P = \frac{N_S}{M(T/60)},$$

where N_C is the number of completed Orders, $N_{C,I}$ is the number of completed integrated Orders, M is the Driver count, and T is elapsed ticks at 60 ticks per hour. The primary productivity metric `service_units_per_driver_hour` has units of completed service units per Driver-hour. At the full 1440-tick horizon its denominator is $24M$. The paired controller reports both $P_{\text{integrated}} - P_{\text{food-only}}$ and the condition-level values needed to calculate the relative change.

The primary environmental metric is emission intensity per completed service unit:

$$EI = \frac{E}{N_S} = \frac{E}{N_F + N_G}.$$

The paired absolute and relative effects are $EI_I - EI_0$ and $(EI_I - EI_0)/EI_0$. A negative value indicates improved emission efficiency. Total emission E remains a secondary outcome because lower intensity does not imply lower absolute emission when integration completes more service output. All values remain model emission units until the per-step factor is calibrated to physical CO₂e.

For a paired integrated condition I and food-only baseline 0, the controller also calculates

$$ME_G = \frac{E_I - E_0}{N_{G,I}}, \quad ME_S = \frac{E_I - E_0}{N_{S,I} - N_{S,0}}.$$

ME_G is marginal model emission per completed grocery service. ME_S is marginal model emission per positive net additional service unit and is left missing when $N_{S,I} - N_{S,0} \leq 0$. The controller also reports percentage changes in service output and total emission and their elasticity $\eta_E = \Delta E\% / \Delta N_S\%$ when defined. With positive output growth, $\eta_E < 1$ means emission grows more slowly than output, which is equivalent to lower emission intensity.

Service level has two dimensions. **Food service protection** covers all generated demand because every order contains food. Its primary metrics are `food_completion_rate` and `p90_food_completion_ticks`. Supporting measures are food cancellation and unfinished rates, mean food completion time, P90 food pickup wait, and mean/P90 time from food handover until delivery at the Customer. These measures are paired against the food-only outcome in the same seeded day.

All food-protection columns in the experiment tables use canonical `food_`-prefixed names: `food_completion_rate`, `food_cancellation_rate`, `food_unfinished_rate`, `mean_food_completion_ticks`, and `p90_food_completion_ticks`. The former generic aliases `completion_rate`, `cancellation_rate`, `active_rate`, and `mean_completion_ticks` are not emitted. Their equality depended on the current all-food demand assumption and obscured the measured population.

Integrated customer experience is restricted to integrated orders. Its primary metrics are `integrated_completion_rate` and `p90_integrated_completion_ticks`. Supporting measures are integrated cancellation and unfinished rates, mean completion time, and mean/P90 food and grocery pickup waits. The controller summarizes these condition-level measures across independently seeded day blocks because the food-only baseline has no integrated orders. All P90 values use NumPy's linear quantile over observed values. On-time completion rate is deliberately absent because no empirical target or SLA is available.

The designated outcome families are productivity, service level, time, revenue and emissions. Primary paired metrics are `service_units_per_driver_hour`, `food_completion_rate`, `p90_food_completion_ticks`, `platform_revenue_per_order`, and `emission_intensity_per_service_unit`; the two integrated-experience primary metrics are summarized separately. All per-order denominators use the actual generated count of the run, not a fixed 300.

Secondary productivity measures are `service_units_per_busy_driver_hour`, which divides by aggregate busy ticks converted to hours, and `service_units_per_km`. Other secondary metrics include Driver and Merchant revenue per generated order, Driver profit per generated order, and emission units per completed order. Service-level secondary metrics use the explicit food names above. The controller also exports `driver_utilization` (total busy ticks/(MT)) and `driver_profit_per_driver` (total settled Driver revenue minus movement cost, divided by M). The last metric is mean daily profit across the whole fleet, including idle Drivers; it is not a distributional or hourly earnings measure.

Counts of completed, cancelled and active orders accompany each run. Read the conditional time means with completion rate and observation counts because different conditions may complete different subsets. Grocery is

absent at $p = 0$, so revenue and emission differences do not compare identical baskets. Emissions retain the model's units; no physical mass unit is inferred. Interpret all outcome families together before judging a setting favorable.

7.6 CLI, Artifacts and Visual Inspection

The controller accepts `--p-integrated`, `--replications`, `--seed-start`, `--demand-csv`, `--output-dir`, `--daily-customers`, `--num-drivers`, `--sensitivity`, `--p-values`, `--food-prep-multipliers`, `--grocery-prep-multipliers`, `--store-selection-values` and `--summary-only`. The four level-list options take effect only with `--sensitivity`; reference values are always included.

```
python -B src/simulation_experiments.py --p-integrated 0.44
--replications 30 --sensitivity --summary-only
```

The command is wrapped for readability; enter it on one line or use the shell's continuation syntax. The current $p^* = 0.44$ is a configurable modeling input, not a calibrated empirical estimate. Current sensitivity defaults execute 360 runs; `--summary-only` avoids seven detailed files per run while retaining all scenarios and experiment tables.

The optional `--visualize-replication` selects a one-based replication; `--visualize-condition` chooses the standard baseline or reference integrated condition, and `--interval-ms` controls playback speed. The visualizer title displays the actual daily count and fleet size, and Reset reuses the selected scenario.

By default, the controller creates a unique directory under `output/experiments`; an explicit destination must be new or empty. It writes `run_summaries.csv`, `paired_differences.csv`, `paired_statistics.csv`, `emission_statistics.csv`, and `service_level_statistics.csv`. It also writes `sensitivity_effects.csv` when alternatives exist, saved scenario JSON files, and `manifest.json`. The emission table summarizes the two marginal-emission measures, output and total-emission percentage changes, and emission-output elasticity. The service-level table summarizes integrated customer experience across day blocks; paired food-protection effects remain in the paired tables. Both use pointwise bootstrap intervals. The paired statistics table additionally records the normality test, selected paired test, statistic, raw and Holm-adjusted p -values, significance decisions, and paired Cohen's d_z . Tables carry condition ID, factor and all four factor-level fields. The pair and statistical tables mark `primary_comparison` only for $p = p^*$ at reference factor settings; `primary_metric` identifies designated outcomes.

The manifest records conditions, reference values, run/scenario counts, seeds, scenario paths/hashes, input and code hashes, generator version, metric names and inferential methods. It declares `experiment_schema_version:8`; this version adds paired significance tests, Holm-adjusted decisions and paired effect size. Version 7 replaced generic service-level aliases with canonical `food_`-prefixed metrics. Engine metadata uses standalone schema 7 and scenario schema 8. Detailed engine exports are under `runs/` unless suppressed. A replay is exported separately and does not enter the experiment tables.

A Review Findings and Disposition

The historical review compared the original 14-page PDF with the source available on 8 September 2026. The current Manhattan source snapshot is identified in Appendix B; export-recovery recommendations below have not been implemented. Severity P2 means a recoverable operational defect that can prevent reliable result preservation. These findings concern exceptional I/O paths, not a demonstrated error in baseline simulation mechanics.

Historical evidence. R1, R2 and D1 below describe the 8 September 2026 review and its source locations. Fault-injection probes were not rerun for the Manhattan change; export-recovery logic is outside this change. Current source locations and tests are in Appendices B–C.

A.1 R1 (P2): Partial Export Directory Prevents Retry

Evidence. `simulation_engine.py`, `IntegratedDeliveryModel.export_results`, lines 797–845, checks for a new/empty destination, then creates it and writes CSV/JSON files directly in sequence. A temporary probe injected an `OSError` opening the second CSV, `orders.csv`, after `kpi_ticks.csv` was written. The destination retained only that first CSV. A subsequent ordinary export to the same path raised `FileExistsError` because it was no longer empty. The model snapshot was unchanged.

Impact. A transient disk or permission failure can leave an incomplete result directory that looks like a run directory and blocks retry, including a visualizer Reset retry using the same explicit destination. The current engine has no all-or-nothing commit or completeness marker.

Recommendation. Stage all seven files in a fresh sibling temporary directory, validate successful completion, and publish the complete set with a no-overwrite finalization step; clean only staging data owned by that attempt. Retain the in-memory run on failure. Until fixed, use a fresh destination for manual recovery and verify all seven files. Merely repeating export to the incomplete directory does not recover it. **Disposition:** documented in §4.3 and §6.3; production code unchanged.

A.2 R2 (P2): Closing After Export Failure Cannot Retry Through Close Handler

Evidence. `simulation_visualizer.py`, `GridVisualizer._on_close`, lines 333–340, sets `is_closed=True` before calling `_save_current_run` and returns immediately on later close calls. `run`, lines 342–353, calls this handler again after `plt.show()` returns, but the guard suppresses another attempt. A temporary probe injected a close-time `OSError`; the first close attempted one export, and a second close attempted zero exports even after the failure injection was removed. The error and closed state were retained; simulation state did not advance.

Impact. The CLI reports an export failure and exits with code 1, but there is no recovery dialog or close-handler retry. An unsaved run can be lost when the process exits. This does not affect successful close-time exports or the intended duplicate-export guard.

Recommendation. Separate playback-closed status from export-success status and provide a bounded retry or fresh-destination recovery before process exit. Address R1 as well so retries do not collide with incomplete directories. **Disposition:** failure behavior is explicit in §4.3; production code unchanged. The probe verifies callbacks headlessly, not a native GUI error dialog.

A.3 D1: Historical Dependency-Version Discrepancy

The requirements file pins Mesa 3.5.1 and NumPy 2.5.2, while the available `food-grocery-integration` environment contains Mesa 3.3.1 and NumPy 2.4.6. README states validation with Python 3.14.7; this review ran with Python 3.11.15. `pandas` 3.0.5 and `Matplotlib` 3.11.1 match. The listed `abms` environment did not expose a usable Python executable during this review.

All 36 existing tests pass on the available environment, but that does not establish byte-identical RNG behavior or compatibility for the declared stack. Record actual exported versions and validate a chosen environment before comparing experiments. Requirements, README and environment packages were not changed.

A.4 Documentation Discrepancies Resolved

The original PDF’s open Merchant-selection, CSV-mapping, tariff-distance and compensation descriptions were replaced by implemented choices in the earlier review. Its algorithm describes one run; this edition additionally documents the implemented experiment controller in §7. Readiness timing, shuffled Drivers, historical assignment, cumulative/observed KPIs, schema versions and visualizer behavior are explicit.

The Merchant diagram checks cancellation first. The interaction matrix includes the grocery-status dependency between Store and Restaurant through the Order and clarifies that Customer does not observe preparation progress. Those 8 September corrections were documentation-only. The 13 September edition changed the distance and movement model described in §1.7–1.8 and §1.16. Python line references in the retained review matrix identify the 13 September implementation; use names and current source for this edition.

B Validation Evidence and Review Snapshot

B.1 Current-Code Validation (19 September 2026)

The complete suite passed 49 tests (32 engine, ten controller and seven headless visualizer tests) in the available project environment: Python 3.12.14, Mesa 3.5.1, NumPy 2.5.2, SciPy 1.18.1, `pandas` 3.0.5 and `Matplotlib` 3.11.1. The engine assertions reconcile completed food and grocery service units and their Driver-hour productivity denominator. The controller checks shared scenarios, dimension validation, food and Driver inputs shared across policies, preparation multipliers, all Store-selection rules and deterministic ties, matching food baselines, paired arithmetic, productivity output, bootstrap intervals, paired test selection, Holm adjustment and visualizer Reset. These are software acceptance checks using illustrative inputs, not empirical findings or sufficient evidence for the final replication count. The dated 17 September and 13 September results below are historical snapshots.

B.2 Paired-Controller Validation (17 September 2026)

The complete engine, controller and headless visualizer suite passed 42 tests in the existing `abms` environment. The three controller tests check shared food inputs and Driver activation across policies, paired CSV arithmetic and bootstrap interval presence, and visualizer Reset with the same scenario. A separate two-replication smoke run at $p^* = 0.5$ with sensitivity points 0 and 1 produced six complete seven-file engine exports, two scenario JSON files, paired differences, statistics and a manifest. These are software acceptance checks using illustrative parameters, not empirical findings or sufficient replication for research inference. The historical 13 September review snapshot follows.

B.3 Method and Result

The following Manhattan-baseline review snapshot is dated 13 September 2026. Its counts, hashes and Python line references identify that historical source, rather than the 17 September paired-controller edition. It inspected matching, movement, tariff and metadata implementation, ran the engine and headless visualizer suites, and rendered the then-current PDF. Historical fault-injection probes P1/P2 were not rerun.

Run from the project root using the available environment:

```
conda activate abms
python -B -m unittest discover -s tests -p "test_simulation*.py" -q
```

Result: 39 tests passed (32 engine tests and 7 visualizer tests). Three new tests cover Manhattan geometry, matching that distinguishes Manhattan from Euclidean/Chebyshev, and actual/billed legs with partial exports and metadata. The orthogonal movement test also checks Euclidean direction, deterministic ties, boundaries, zero-distance arrival, waits, cost and emissions. Existing tests cover lifecycle, exact horizon, finance reconciliation, seed reproducibility and playback equivalence. These are implementation checks, not calibration or native GUI acceptance tests.

Dependency	Declared / README claim	Actually tested
Python	README: 3.14.7; script: ≥ 3.11	3.14.7
Mesa	Requirements: 3.5.1	3.5.1
NumPy	Requirements: 2.5.2	2.5.2
pandas	Requirements: 3.0.5	3.0.5
Matplotlib	Requirements: 3.11.1	3.11.1, Agg test backend

One-day acceptance runs use seed 42 and the default CSV. Each produces 1440 KPI records and 300 orders and passes status, movement and financial reconciliation. Results below are illustrative checks for the new baseline, not performance targets or multi-replication estimates. Active includes AVAILABLE, ASSIGNED and DELIVERED awaiting acknowledgement; delivered cumulative includes completed orders.

p_I	Completed	Cancelled	Active	Delivered cum.	Travel km
0	104	189	7	105	2702.0
0.5	85	206	9	85	2742.0
1	71	219	10	71	2740.0

Restaurant and Store PREPARING-cancelled-unit aggregates are zero in all three runs. Identical input, seed and runtime reproduce the new baseline; old diagonal-baseline outputs are not expected to match.

B.4 Content Hashes

These SHA-256 values identify the 13 September reviewed bytes; line references below apply to that historical snapshot. The 17 September controller manifest records current engine/controller bytes for each experiment. Updating or rebuilding the PDF changes its hash. The original PDF is preserved as `backup/Integrated_Food_Grocery_Delivery_Model_Documentation.pre_review_20260908.pdf`. The immediate pre-Manhattan PDF is separately preserved as `backup/Integrated_Food_Grocery_Delivery_Model_Documentation.pre_manhattan_20260913.pdf`.

Artifact	SHA-256 of review input
Engine	c5ca651beac6e6ae528e80e44ca79211b1839a5c9a8308e1abc8abc7ff1e2512
Visualizer	65b73fbdc8f322db691828c1913eafefda285e8550a525962ce28fecc434caa4
Engine tests	439269e06800ca35b6e890f0df85145c4b4743208dbf0290b2efdcc1a80c0a1f
Visualizer tests	e99e1d8fc6d04420d287746a414778f6a6b5a3ed92e6e5cecb0898cb01e252e1
Original PDF	ff007295371d70f4425006951e00656fcae71c3136999a5f9a1355931f087b01
Demand CSV	04feff3a2417f606d3d9204bf5400cf0e4da350b9f3aff15a85722b660517333

B.5 Existing Test Index

E denotes the engine test module and V the visualizer test module, both under `tests/`. Each location identifies an existing test method. The matrix uses these evidence IDs.

ID	Location	Test method
E1	E: 51–59	<code>test_probability_required_and_validated</code>
E2	E: 61–70	<code>test_hourly_average_and_largest_remainder_ties</code>
E3	E: 72–77	<code>test_divmod_can_spawn_multiple_customers_at_same_tick</code>
E4	E: 79–89	<code>test_invalid_csv</code>
E5	E: 91–94	<code>test_large_finite_weights_do_not_overflow</code>
E6	E: 96–105	<code>test_baseline_populations_and_no_order_agents</code>
E7	E: 107–115	<code>test_normal_parameters_clamping_and_ceiling</code>
E8	E: 119–149	<code>test_integrated_colocated_timeline_and_no_same_tick_state_chaining</code>
E9	E: 151–166	<code>test_preparation_zero_one_and_parallel_records</code>
E10	E: 168–188	<code>test_timeout_precedes_matching_at_exact_threshold</code>
E11	E: 190–198	<code>test_assigned_orders_never_timeout</code>
E12	E: 200–209	<code>test_customer_cancellation_wins_over_becoming_ready_same_tick</code>
E13	E: 211–227	<code>test_ready_cancellation_disposal_vs_return_recorded_once</code>
E14	E: 229–252	<code>test_every_handover_guard_and_grocery_precedence</code>
E15	E: 254–264	<code>test_pickup_wait_counts_only_unsatisfied_pickup_activations</code>
E16	E: 266–285	<code>test_resolving_previous_order_preserves_new_driver_assignment</code>
E17	E: 287–303	<code>test_last_tick_delivery_is_not_acknowledged_or_settled</code>
E18	E: 307–329	<code>test_matching_distance_then_fifo_then_id</code>
E19	E: 331–343	<code>test_integrated_matching_uses_store_not_restaurant</code>
E20	E: 373–417	<code>test_movement_steps_orthogonal_cost_and_boundary</code>
E21	E: 465–489	<code>test_billable_legs_quotes_and_exact_settlement</code>
E22	E: 493–524	<code>test_integrated_components_evaluated_independently</code>
E23	E: 526–539	<code>test_active_and_completed_orders_do_not_count</code>
E24	E: 541–589	<code>test_cumulative_zero_value_and_repeated_partial_exports</code>
E25	E: 598–620	<code>test_daily_reconciliation_and_permanent_merchants</code>
E26	E: 622–631	<code>test_seed_reproduces_complete_history_and_shuffle</code>
E27	E: 633–639	<code>test_probability_endpoints</code>
E28	E: 641–660	<code>test_export_reconciliation_and_no_overwrite</code>
E29	E: 662–689	<code>test_empty_partial_export_and_detached_snapshot</code>
E30	E: 345–353	<code>test_manhattan_distance</code>
E31	E: 355–371	<code>test_matching_uses_manhattan_not_straight_line_or_diagonal_steps</code>
E32	E: 419–463	<code>test_actual_legs_partial_export_and_billable_distance</code>
V1	V: 28–40	<code>test_render_snapshot_and_speed_do_not_change_time_or_rng</code>
V2	V: 42–53	<code>test_pause_step_and_resume_advance_exactly_one_tick</code>
V3	V: 55–70	<code>test_reset_restores_seed_and_preserves_partial_export</code>
V4	V: 72–84	<code>test_close_exports_partial_once_without_advancing</code>
V5	V: 86–95	<code>test_export_failure_is_visible_and_reset_keeps_model</code>
V6	V: 97–113	<code>test_changed_demand_blocks_reset_without_saving_stale_results</code>

ID	Location	Test method (continued)
V7	V: 115–135	<code>test_visualized_full_run_matches_engine_and_exports_at_horizon</code>

C PDF-to-Code Traceability Matrix

Locations use E = `simulation_engine.py` and V = `simulation_visualizer.py` in `src/`. Status: A = aligned, D = outdated text corrected, C = missing detail completed, X = conceptual feature outside the implementation, B = reproduced software defect. Evidence IDs refer to the existing test index; S means direct static inspection, and P1/P2 are historical R1/R2 probes from 8 September, not rerun for this edition. Sections in the first column identify the original PDF unless marked new.

PDF topic	Implementation and source location	Review disposition / documentation action	Evidence
Notation / Algorithm 1	E: <code>SimulationConfig</code> 95–110; run 581–584; CLI 857–891.	D/X: one invocation, fixed horizon and populations; remove implemented outer experiment-loop claim. Rename replication notation to avoid collision with Restaurant set. Required probability and default seed are explicit.	E1, E27
1.1 Activation	E: model initialization 453–474; <code>step</code> 561–579.	C: exact Customer–Store–Restaurant–shuffled Driver order; initialization/RNG sequence and end-of-tick clock. Timeout precedes matching; no horizon drain.	E10, E26, E17
1.2–1.3 Space / agents	E: constants 38–40; <code>_random_position</code> , <code>_initialize_agents</code> 480–489; <code>_resolve_orders</code> 546–559.	A/C: non-toroidal MultiGrid, fixed populations, permanent Merchants, temporary Customers; Orders/components are records rather than agents.	E6, E25
1.4 Demand	E: <code>load_hourly_weights</code> 113–133; <code>build_demand_schedule</code> 136–149.	D/C: specify default CSV, required columns, per-hour mean and normalization, hour tie rule and random distinct minute remainders; preserve exact total and multiple spawns.	E2, E3, E4, E5
1.5 Integration	E: <code>_create_order</code> 494–527.	D/C: independent uniform Merchant selection; required Bernoulli probability. Every Order contains food; no independent grocery demand or merging.	E7, E27, S
1.6 Statuses	E: enums 54–91; records 174–244; active/history collections 458–460.	A/C: separate Order, component, Merchant and Customer states; clarify terminal component records and active-versus-historical membership.	E6, E25
1.7 Matching	E: <code>Driver._claim_order</code> 373–389.	A/C: Driver-owned Manhattan/FIFO/ID selection to first Merchant, immediate claim and one order per Driver; no dispatcher or acceptance probability.	E18, E19, E31
1.8 Movement	E: <code>manhattan</code> 152–154; <code>Driver._move</code> 391–409.	A/C: squared Euclidean argmin, deterministic coordinate tie rule, four orthogonal neighbors/current cell, actual-move accounting. Movement count is distinct from service duration.	E20, E8, E30
1.9 Driver states	E: <code>Driver.step</code> 411–449; <code>_resolve_orders</code> 546–559.	D/C: release operational order at DELIVERED, retain historical Driver ID, and protect a new claim during old-order resolution. Exactly one initial-state branch per activation.	E8, E16
1.10 Preparation	E: <code>planned_ready_tick</code> 189–190; sampling 491–508; Merchant readiness 304–307.	A/C: preserve Normal parameters, ceiling/clamp and per-component draws; explicitly define creation-plus-duration readiness, including zero and cancellation precedence.	E7, E9, E12
1.11 Merchant	E: <code>Merchant.state</code> 279–282; <code>step</code> 294–309.	A/C: derived IDLE/ACTIVE, parallel preparation without capacity, all Merchants visited; terminal records skipped. Conceptual methods are not Python public methods.	E9, S
1.12 Handover	E: <code>Merchant._can_handover</code> 311–322.	A/C: all six guards, historical assignment versus current operational order, and grocery-before-food; arrival alone cannot hand over in the same tick.	E14, E8
1.13 Customer	E: <code>Customer.step</code> 253–266.	A/C: status/reference-only observations; completion occurs at next Customer activation. No preparation/Driver-position tracking.	E8, E17, S
1.14 Cancellation	E: <code>Customer.step</code> 253–266; <code>cancellation_kpis</code> 285–292; <code>_handle_timeout</code> 324–332.	D/C: PREPARING units/item value replace compensation fields; READY disposal/return excluded. Component-independent, idempotent event accounting; no payment posting.	E10, E11, E12, E13, E22, E23, E24

PDF topic	Implementation and source location	Review disposition / documentation action	Evidence
1.15 Prices	E: <code>_normal_nonnegative_integer</code> 491–492; component creation 503–508.	A: preserve distributions, zero clamp/ceiling, integer rupiah, and zero grocery value for food-only. Keep modeling-assumption qualification.	E7, E21
1.16 Tax / tariff	E: <code>_create_order</code> 514–522.	D/C: Manhattan billable legs and km units; exclude first pickup travel; clarify fractional distance and creation-time quotes rather than payment events.	E21, E32
1.17 Cost / emission	E: constants 42–44; <code>_move</code> 391–409; KPI calculation 586–650.	A/C: include all actual cell changes, even on unfinished orders; no stationary cost/emission or invented mass unit.	E20, E25, E32
1.18 Finance	E: <code>merchant_revenue</code> 157–161; <code>_settle_order</code> 529–544; <code>_driver_row</code> 675–693.	A/C: once-only completed settlement, Decimal text, residual Platform revenue, all-travel Driver cost; no Merchant/Platform profit or cancellation compensation.	E21, E16, E17, E25
1.19 KPIs	E: <code>Order.time_metrics</code> 232–244; <code>_calculate_kpis</code> 586–650; record builders 652–708.	D/C: observed means/counts, null values, completion total, pickup waits so far, utilization, cumulative delivery, status/finance reconciliation and cancellation KPIs.	E15, E25, E24, E29
1.20 Open items	E: <code>SimulationConfig</code> 95–110; creation 494–527; metadata policies 760–793.	D/X: distinguish experiment inputs from resolved CSV, Merchant, tariff and cancellation choices; multi-run orchestration and compensation payments are unimplemented.	E1, E28, S
1.21–1.22 Interpretation	E: agent steps and shared dataclasses; V: snapshot-based drawing 199–230.	C: keep no-Dispatcher baseline; separate descriptive implementation from empirical validity and unresolved software defects; explain PDF hash changes.	S
Figures 1–3	E: <code>Customer.step</code> 253–266; <code>Merchant.step</code> 294–309; <code>Driver.step</code> 411–449.	D/C: native TikZ diagrams show initial-state branches, delivery/ack separation, derived Merchant state, terminal skip and cancellation-before-readiness. Existing diagram style retained.	E8, E12, S
Interaction matrix	E: <code>_create_order</code> 494–527; <code>_can_handover</code> 311–322; Customer 253–266.	D/C: distinguish shared data from physical handover; add Store-to-Restaurant grocery-status dependency and remove implication of Customer preparation observation.	E14, S
New: visual display	V: <code>DRIVER_STYLES</code> 33–40; constructor 46–125; display/dashboard 139–230. E: <code>snapshot</code> 719–735.	C: document symbols/colors, display-only offsets, endpoint connectors, visible KPI subset, initial/last-tick clock and rounded display.	V1, V7, S
New: playback	V: finish/advance/frame callbacks 251–272; pause/step 274–296; speed 326–331.	C: exact one-tick advancement, independent animation speed, pause behavior and horizon stop/export. Rendering does not introduce model decisions.	V2, V1, V7
New: reset / close	V: <code>_save_current_run</code> 232–249; <code>_reset</code> 298–324; close/run/main 333–376.	C/B: validate candidate demand before export/replacement, preserve model on failure, sibling output naming, partial close and success cache. Explicitly document R1/R2 limits.	V3, V4, V5, V6, P1, P2
New: API / CLI	E: config 95–110; run/summary/snapshot/export; arguments 857–891. V: constructor 46–68; main 356–372.	C: document existing signatures/flags, validation, defaults and fixed baseline; model.run returns summary without exporting. CLI unchanged; Manhattan helper replaces Chebyshev.	E1, E29, S
New: exports / schema	E: <code>_metadata</code> 737–798; <code>export_results</code> 800–848; <code>_json_default</code> 851–854.	C/B: seven-file catalog, schema 2, distance policies, provenance, Decimal/null encodings, stable empty headers and no overwrite. Qualify incomplete-write atomicity (R1).	E28, E29, E24, P1